Weekly Goals

- Finish weather generation
    - Finish mapping
    - Finish preprocessing
    - Combine Data
    - Generate new Training Data
- Update Single Source of Truth
- Look into AI model scalability

I believe after this, there is 1-2 more weeks of development before the project needs to be presented. At present I don't want to undertake any new assignments unless I know that they are critical. I want to begin putting together documentation about what has and has not been done as well as addressing some of the pitfalls or things that can still be improved upon, some of which I do not have an answer for yet.

An ideal week would be to have the data finished before the meeting on 4/23/2026 and having a proposal for scaling the model, which I think can borrow the same system from the weather API of caching for 10 minutes. I would want to look deeper into what the model takes in and what it returns as well as what the ideal take in/return is.

At this point I do not believe that I will take on processing the large CSV file from NOAA. I am willing to try and make progress but unless the weather becomes much simpler to begin on than I currently think it will (and historically has shown itself to be) then I don't think I will have time to get that done and anything else of note. It would be a good way of showing how well the model works and I am sure we could come up with some way of demonstrating accuracy based off of the model working with roads it has not seen yet, but I believe that we will not have time for it. Another month and I am sure I could finish, but not yet.

I write this just to get my thoughts on paper about what I think is reasonable about my task work moving forward. At the end of the day, I am somewhat solo on what I am doing and if most other issues revolve themselves, then I am open to working on whatever.

Weather Generation

Location Mapping

On the week 8 report I was last going over how to scale the ability to map stations at a county level. While I do generally have a plan for that, I am going to pivot slightly and move onto just getting the function done for a city level, then work on preprocessing, then move onto making them all compatible with county level work. My week 8 report was finished while I was low on sleep and rambling.

The last pitfall from week 8 was the rate limiting from Nominatim (https://nominatim.org/), Putting in a simple function to sleep the queries one second between seems to do the job at preventing rate limiting. Now when we send a request to the following site: https://nominatim.openstreetmap.org/search along with the city name, state name, and country, we can get the appropriate information back:

```
{'lat': 32.5888072, 'lon': -96.9553675, 'display_name': 'Cedar Hill, Dallas County, Texas, 75104, United States'}
```

From here, an array of these values can be stored and inserted into stations.csv such that the latitude and longitude are compared with the values that already exist inside of stations.csv. Once a comparison is made, the station with the smallest difference in both latitude and longitude will be put in the slot for closest_station and the station_flag will be set to 0. For reference a pre-existing entry looks like this.

```
Addison,32.969,-96.836,Addison,1
```

The mapping is now complete, each city that has a name that appears in geocaching now has a ‘closest station’ that it will pull weather from during weather combination (combining weather data from multiple sources with crash samples). Duplicates have been removed which result in only one case that still needs to be accounted for.

```
Could not geocode 'OUTSIDE CITY LIMITS', skipping.
'DALLAS' already has an entry in weather/weather_csvs/stations.csv, skipping.
'MESQUITE' already has an entry in weather/weather_csvs/stations.csv, skipping.
'LANCASTER' already has an entry in weather/weather_csvs/stations.csv, skipping.
'IRVING' already has an entry in weather/weather_csvs/stations.csv, skipping.
'ADDISON' already has an entry in weather/weather_csvs/stations.csv, skipping.
Appended 24 row(s) to C:\Users\DJKei\Ai Road
```

With 30 cities and 5 already existing, 29 have been added to the stations list. This leaves just ‘OUTSIDE CITY LIMITS’ as a data point to map to. While this will not account for much of the data, it should still be accounted for in the data. Thus, my general solution for now will be to attach ‘OUTSIDE CITY LIMITS’ to the first station that exists in the list. ‘OUTSIDE CITY LIMITS’ or any other cities that for some reason cannot be geocoded will be appended in this way and will be attached without coordinates. Two samples are below.

```
COCKRELL HILL,32.7362421,-96.8869481,Dallas,0
OUTSIDE CITY LIMITS,,,Mesquite,0
```

While doing this, I have come to the realization that there is a fatal error if there are no stations that exist, then no weather can be made. I see this as a fringe case, but one that I need to correct before i can move on to preprocessing. This needs to be added to the matching step (csv_lookup.py) and the download step (weather_download.py).

If csv_lookup.py does not download any stations from stations.csv, False will be returned and the CreateTrainingData.py file will be aborted. If weather_download.py does not append any stations. Stations.csv false will be returned and the same abortion will occur.

Preprocessing

csv_weather.py was already built before better weather coverage needed to be met. This is made of six steps: removing unnecessary columns, standardizing date and time, removing rows with error values for precipitation (null, empty, whitespace, or 9999.99 which is a common debugging value), tracking ice, removing rows without ice flags or precipitation, then combines all the data into a single FinalWeather.csv.

Most of this file can remain unmodified however, the following things need to be worked on:

- Looping through weather_csvs folder for valid csvs
- Outputting correct final weather
- Ensuring that any files generated are reused and then cleaned up correctly.

The first step I will solve is looping through the files inside of weather.csvs, saving an array of the file names to be used to update them for final weather. stations.csv will also need to be filtered out of this list. This list is returned:

```
['Addison_2025.csv', 'Dallas_2025.csv', 'Irving_2025.csv', 'Lancaster_2025.csv', 'Mesquite_2025.csv']
```

These are all the csv files that we will need to loop through and preprocess. In the Dallas subset, these are correct and it does not include the stations.csv that exists in the same directory.

I realize after that it would almost be easier to do preprocessing during the download stage. If this needed to be queried multiple times over and over again or for the applications function, I would likely rework the download stage to have this preprocessing step, however data generation needs to be done very few times by a single person or two and so I believe that the best step is to keep it separate if for nothing else but speeding up the testing phase.

For now we need to define the new way the csv_weather.py will work. The beginning of the file takes the START_WEATHER_FILE and the FINAL_WEATHER_FILE. These can be combined to be a singular file. For now I will attempt to have this place data directly into the weather_csvs directory. Given that I want this to be a singular function that can be run from CreateTrainingData.py, I will modify the main function of csv_weather.py to be a function that first pulls all the file names then loops through all the names.

While going through preprocessing step by step, I have noticed that while Dallas is processed correctly, some of the other weather files are reduced down to 200-30 lines out of their original 10,000 lines. Given that we need 8,760.lines (for hours in a year), all the files start with that amount. I assume they are holding precipitation differently and the error comes from the 'remove_empty_aa1' step.

The issue does seem to be the aa1 step. With the weather format that I am using, Mesquite, Lancaster, and Addison, seem to have almost no AA1 data which I had taken to be precipitation. Instead of Dallas, most of them seem to keep track of 24 hours of precipitation at a time instead of noting every single hour (unless something fantastic seems to happen. Another step is probably going to need to be added that splits the 24 hour tracking into 24 individual rows.

The remaining problem is rows like this:

```
DATE,VIS,TMP,AA1
2025-08-02T05:59:00,"999999,9,9,9","+9999,9","24,0058,9,6"
```

Where it has precipitation (58 mm across 24 hours) which is workable, but has no usable temperature or visibility. Most of the csv files containing information have rows like this. Originally when making these I took these to be 'debug times' or times where the station would output something as Dallas was and is very well documented.

However, stations like Addison have barely any data if we assume that these are debug times.

Reevaluation

My goal was to create a way to automate weather gathering. NOAA was what this was based off of and up until now I believed it would work for the purposes I needed it to. I have made several assumptions to make it work such as being okay with it only going to mid-August or only having 5/29 (not including Out of City Bounds) but this is catastrophic to actually representing good data. The stations represented would need to go down to 2-3/30 which is no better than having one station represent everything.

I truthfully do not think that I can in good conscience move forward with using NOAA. I would need to assume that any time we got that block of 999999 temp and visibility that the whole 24 hour period was clear and sunny which I cannot in any good conscious do. I would also need to design a system to figure out when duplicates are, which I am sure I could do, but is a lot of overhead. I have spent several weeks trying to get this system automated and while I am very disappointed that I cannot continue using NOAA and will need to scrap it, I think that is the correct move.

I had the thought of not automating and just manually downloading all the files, but the download step already does that. Meaning that there likely is no better data from NOAA. I have on several occasions promised better data and I do intend on delivering it. There are other forms of data that I am sure I can use. Open-Meteo is my first choice and I will look into using that instead. If it is similar to the data it provided for the weather api, preprocessing will be even more simple than the NOAA data, the only concern I immediately have for it is that without paying for access, download times may be awful or impossible.

Open-Meteo does seem to have a way to access that information with a single api call.

Open-Meteo

An api call to their archive will return this type of csv format:

```
latitude,longitude,elevation,utc_offset_seconds,timezone,timezone_abbreviation
32.724075,-96.48352,134.0,0,GMT,GMT
```

```
time,temperature_2m (°C),precipitation (mm),weather_code (wmo code)
2025-01-01T00:00,10.2,0.00,0
2025-01-01T01:00,9.4,0.00,0
2025-01-01T02:00,8.2,0.00,1
```

The first 3 lines will need to be removed, but this is proof that Open-Meteo should be able to provide an acceptable substitute for NOAA in terms of weather. I believe that Open-Meteo is going to be less generally accurate when it comes to the weather that it is reporting, but given that NOAA has almost no coverage, I believe Open-Meteo is an acceptable alternative.

Now that I know Open-Meteo is able to produce weather, we need to plug it into the general setup. I am going to continue the general structure of pairing it up with cities. Thus, the first step is to get the latitude and longitude of all the cities and then feed that list into this file. Fortunately, this function already exists inside of csv_geocode.py so only a slight modification to that file will need to be done.

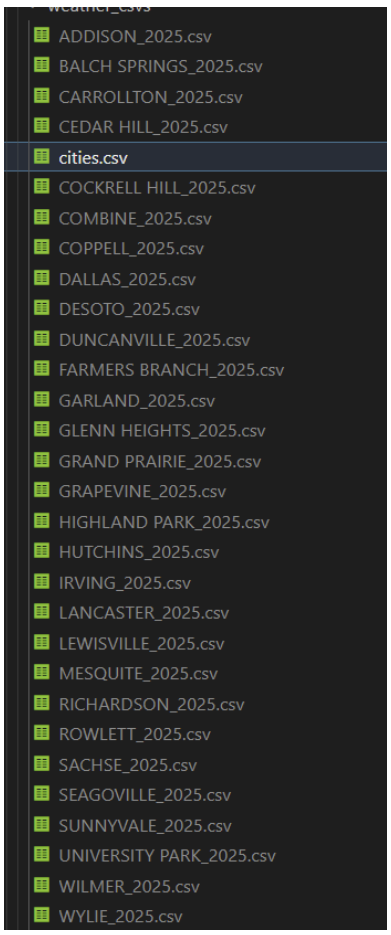
csv_lookup.py has been modified (only slightly to be able to get the latitude and longitude of all cities mentioned by whatever the main crash file has given. It is essentially a more brief version of stations.csv:

```
name,lat,lon
UNIVERSITY PARK,32.8551267,-96.7919454
DALLAS,32.7762719,-96.7968559
```

The next step is going to be to simply ensure that Open-Meteo can do what we need it to do and produce a csv file for all 29 pairs of latitude/longitude coordinates. If it can produce at least 5 usable files, then it will work as a successful alternative to the NOAA data.

There does appear to be 29-30 files that have been generated. Thus, I will move forward with NOAA. The last thing that needs to be done before moving to preprocessing of this new data is to ensure that all the files are named correctly.

With Open-Meteo the following files are now generated and all should have appropriate data attached them.



Open-Meteo Preprocessing

The data that has been downloaded from open-meteo is in this format:

```
time,temperature_2m (°C),precipitation (mm),weather_code (wmo code)
2025-01-01T00:00,9.8,0.00,0
2025-01-01T01:00,8.8,0.00,1
```

Time can be broken up into date and hour. Temperature can be kept as is as can precipitation (though upon reflection it may not be a needed column). Weather code can create the weather condition and code can be reused from the weather API made from week 7. This turned the same weather code into active weather and road conditions.

The only thing that will need to be added is an 'ice flag' same as the NOAA preprocessing. This will track ice and snow staying on roads as well as slush on the roads. Despite the fact that the weather API cannot track similar conditions, it is good to have them in the training data for when the backend is able to track ice and slush on the road.

```
date,hour,temperature_2m (°C),precipitation (mm),weather_code (wmo code)
```

After splitting date and hour this is the resulting way the csv file is ordered. Where date is year-month-day and hour is an int 0-24. The next thing to do is to add an ice flag. The original logic I used to create the initial training data can be reused.

From my week 3 report:
"If the temperature is 0 degrees or less and there is precipitation, make the ice flag set to 1. If the flag is 1 and less than zero, we can assume ice or snow. If above 0 we can assume slush. The flag will continue to be in effect until 5 degrees celsius (about 40 degrees fahrenheit) which is a safe assumption to make.

The logic is, if freezing and raining, all flags from here until the temperature rises above 5 degrees celsius should be flagged as potential ice or winter conditions on the road."

In addition, the weather code will notify us that snow exists. For now, I will simply add the ice flag, if inclement weather is under 0 degrees celsius, the ice flag is set to 1 and disappears when the temperature rises above 5 degrees celsius (41 degrees fahrenheit). This is not on its own a perfect arrangement, but it is a reasonable assumption in my opinion

The ice_flag addition produces this csv structure along with sample data

```
date,hour,temperature_2m (°C),precipitation (mm),weather_code (wmo code),ice_flag
2025-01-01,0,9.6,0.0,0,0
```

Unlike the previous weather preprocessing, no guessing at what the weather is needs to be done, Open-Meteo has a clear list of what the weather is.

Code	Description
0	Clear sky
1, 2, 3	Mainly clear, partly cloudy, and overcast
45, 48	Fog and depositing rime fog
51, 53, 55	Drizzle: Light, moderate, and dense intensity
56, 57	Freezing Drizzle: Light and dense intensity
61, 63, 65	Rain: Slight, moderate and heavy intensity
66, 67	Freezing Rain: Light and heavy intensity
71, 73, 75	Snow fall: Slight, moderate, and heavy intensity
77	Snow grains
80, 81, 82	Rain showers: Slight, moderate, and violent
85, 86	Snow showers slight and heavy
95 *	Thunderstorm: Slight or moderate
96, 99 *	Thunderstorm with slight and heavy hail

For now I will copy the backend weather api conversion into the preprocessing pipeline. I do think I will change 1 setting the weather to cloudy to clear, given it is mainly clear (and testing the weather api in real world conditions) but that change won't immediately reflect to the weather api.

Before this step is done, I will remove any rows that have a weather code of 0 or 1 and no ice_flag as they can be assumed to be the 'default' or clear skies and roads. During the same pass I will update the weather.

For road conditions, I can now actually differentiate between snow and ice. If the weather is snow, set road conditions to '4 - Snow', then '5 - Slush' when above 0 degrees. If there is no snow, set to '6 - Ice' until weather is above 2.5 degrees then set to '5 - Slush'

This has turned the csv data into this structure (with sample data)

```
date,hour,temperature_2m (°C),precipitation (mm),weather_code (wmo
code),weather,ice_flag,Road
2025-01-01,2,8.8,0.0,2,2 - CLOUDY,0,1 - DRY
2025-01-01,3,8.6,0.0,2,2 - CLOUDY,0,1 - DRY
2025-01-01,4,8.0,0.0,2,2 - CLOUDY,0,1 - DRY
```

I would like to remove temperature, precipitation, and weather_code. However, at this point I am on a time crunch to try and get data produced before the meeting on 4/23/2026. I don't believe I will get the data done, but I will attempt to do so. Furthermore, it is unlikely that I will be able to do a dive into the scalability of the model, but I do have some ideas that I will note later on in the document.

Data Combination

Before general clean up of my files, data, and directory, I wish to fully combine the crash data and the new weather data generated.

From the weather files I need to reference date and hour to collect weather and road. The final training data holds these values in the csv:

```
Crash ID,City,County,Crash Date,Crash Month,Crash Time,Crash Year,Day of Week,Hour of
Day,Road Class,Rural Urban Type,Street Name,Surface Condition,Weather Condition,Crash
```

Surface Condition and Weather Condition are the values that come from the new weather. At present, the file that produces the training data csv_editing.py expects there to be only 1 weather file. The task now is to get it so that each road is mapped to a city, then when negative data is generated, the appropriate city weather file is searched for the corresponding hour. If the hour/date does not exist, then set to "1 - CLEAR" and "1 - DRY"

Upon reflection, each city is stored when creating the road reference sheet and so the only step left is to modify the negative samples.

It is almost impossible to verify that the training data generated correctly by simply combining over all 3.9 million lines of training data, but the data has been successfully generated. It has yielded this general structure.

```
City,County,Crash Date,Crash Month,Crash Time,Crash Year,Day of Week,Hour of Day,Road
Class,Rural Urban Type,Street Name,Surface Condition,Weather Condition,Crash,Crash ID
```

Crash ID is the one thing that is not supposed to be here. I am going to remove Crash ID and then generate training data from the beginning of the pipeline all the way to the end. This will intersect the 4/23/2026 meeting time, but it will have to do.

The only other thing that needs to be done, as requested of me, is have some data the separates Street Name and Segment. For the training data generated above, I kept the 2 together so that the AI could not create a link between unrelated road segments. However, it was requested of me and so I will do so.

4/23/2026 Meeting

I provided a first set of new training data during this meeting just to show that I could. From here I have been requested to send training data with sections being separate from street names as well as a list of coordinates of all the sections of roads. This will be my first priority to get done.

Next I am going to clean up the data and commit documentation. This will largely be undocumented here and instead will be reflected inside of the github repo as well as the single source of truth. For the single source, given that I have redone the pipeline of training data generation some of the old data will need to be removed/reworked to model after the requests and data types that I have used for this final version.

I have also been asked to look into the model. This includes both implementing the weather API into the model endpoint. Ben, the man who worked on the model, seems to have a vision for how the weather API should be integrated that I don't feel I fully understand, but I will give it a try. I also intend to look into what goes into the model. My thought process is that some degree of scaling can be done through caching. As I understand it, the two most changing aspects that the model takes in is weather and road conditions which are already cached in 10 minute bursts. Theoretically roads could be cached in the same 10 minute bursts. I am hesitant to do anything more to the model given Ben still seems to be in active development though, but I will be sure to bring it up at the next meeting.

Finally the group is going to begin to look into generally testing the website. Given that the end is coming up, work on the final report, looking into the template, and testing the website is going to begin this upcoming week. I will likely not be able to address this during this week, but it will be among my higher priorities in the upcoming week.

I will check with the professor/TA on 4/24/2026 to see how critical getting the C.R.I.S. bulk data is as it is the way we will test for general success of the model in an ideal world. I believe we have enough to show that the model generally works, but we could come up with some way to justify success.

Finishing data

Providing Final Data

The two files I need to provide is a list of latitude and longitude coordinates for the different sections of the road in order to help make bounding boxes and the same training data but with section and street name as 2 distinct coordinates.

The first set of data is simple to generate and just involves adding coordinates to the RoadReference.csv. The main issue is that the coordinates from the crash data that the road names are pulled from don't necessarily equate to the center of the road section.

For now, the first thing I am going to do is add the section column to the RoadReference.csv. For now it tracks the follow data:

```
Road Name,Rural Urban Type,County,City
```

I will remove Rural_Urban_Type which was mostly a way of logging and testing to see how many roads had that type of flag. Keeping track of sections will be a better use of the space. I will need to treat the two as an identifying pair such that the generate_negative_samples function does not skip over sections of roads.

```
Road Name,Section,County,City
```

From there it is just getting the create_negative samples to take each pair of Road Name and Section as an individual street in order to make the next set of training data.

```
City,County,Crash Date,Crash Month,Crash Time,Crash Year,Day of Week,Hour of Day,Road
Class,Rural Urban Type,Section,Street Name,Surface Condition,Weather Condition,Crash
```

The next thing I need to produce is a list of all the roads with sections and their sections. From there I have been asked to provide the center of the road segment/section. From there I have been told that the general mapping/bounding box tool should be able to generate what needs to be generated on its own.

Road Segment Mapping

To collect the coordinates of all the road segments that exist, I do want to do another pass through the data, creating a second kind of 'RoadReference.csv' that is made up of all the road sections. RoadReference.csv is created after all roads with less than the average amount of crashes are filtered out. I do not believe that any roads with sections have been filtered out as those are usually highways which have a large amount of crashes in comparison to any other types of roads, but I believe it is best to make another run to ensure that is not a needless assumption that I am making.

This has resulted in 52 distinct roads with sections kept track in a file called RoadSections.csv. This next step would be to collect the center of the road.

An immediate solution would be to collect all the crash coordinates and then take the average of all those crashes. This would come up with something close, but I do want to create concrete data.

At present the only technologies that I have located require the downloading of potentially gigabytes of data which I won't do for such a simple request. Instead I will go ahead and use the RoadSection.csv which will then go through and collect all the latitude and longitude and average them.

This is not ideal, but is acceptable for now. It will require generating new data from C.R.I.S. that includes latitude and longitude

This is the new type of data that needs to be downloaded from C.R.I.S.

```
"Crash ID","City","County","Crash Date","Crash Month","Crash Time","Crash Year","Day of
Week","Hour of Day","Latitude","Longitude","Road Class","Rural Urban
Type","Section","Street Name","Surface Condition","Weather Condition"
```

I will send the data over and hope that it works. If not I will come back to this next week and go through another attempt.

9 Dennis Keithly

Work to be done (either before the entire project deadline or to be noted)

One of the things I wanted to do this week was to track all the things that I had mentioned wanting to and did have the time to do or things that I was generally curious about. This is by no means a comprehensive list, but with the end of the project approaching, these insights may be useful for a final report:

NOAA bulk csv processing

Cutting down the amount of negative samples thrown for the training data (remove the issue of 50k to 3M) to save space

Scaling the AI model to some degree

Looking into how fast a paid version of Open-Meteo could run with the weather API

Making the weather API track melting ice/wet roads better (no instant melt ice)

Next Week's Goals

- Ensure that the road segment mapping is adequate and restructure data if it is not
- Integrate weather api with the model prediction api
- Clean up general csv code and merge with main
- Update SSoT with new csv pipeline information
- Begin testing the deployed website and looking for bugs
- If time permits begin looking to caching model output